

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 References

2. Analysis Model

- 2.1 Methodology Used (Object-Oriented + Agile)
- 2.2 Use Case Diagram (with Specifications)
- 2.3 Activity Diagram
- 2.4 Sequence Diagram
- 2.5 Class Diagram

3. Design Model

- 3.1 Architectural Design
 - 3.1.1 System Architecture Diagram
 - 3.1.2 Description of Architectural Design
- 3.2 Database Design
 - 3.2.1 Data Dictionary
 - 3.2.2 Normalization
- 3.3 Component Design
 - 3.3.1 Package Diagram
 - 3.3.2 Component Diagram
 - 3.3.3 Deployment Diagram
- 3.4 Interface Design
 - 3.4.1 Screen Descriptions

Appendix C1: Glossary

Appendix C2: To Be Determined List

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) describes the complete architectural and functional design of UniCare – an AI-powered complaint and issue resolution platform developed for academic institutions. UniCare enables Students to submit complaints digitally, which are automatically classified by category and priority using an integrated AI microservice. The system allows Administrators to review, resolve, reassign, and escalate complaints through a dedicated management panel, thereby replacing manual and paper-based grievance processes with an efficient, traceable, and data-driven workflow.

This document covers the full design of UniCare Version 1.0, including analysis models (use case, activity, sequence, and class diagrams), architectural design, database design, component design, and interface specifications. It serves as the primary technical reference for developers, database engineers, testers, and faculty evaluators throughout the development lifecycle.

1.2 Document Conventions

UML notation is used for all diagrams in this specification. Bold text denotes headings or key terms. Italics denote technical terms, class names, or system identifiers. Table headers appear with shaded navy backgrounds. Class and component names follow PascalCase convention. Database field names use snake_case. All REST API endpoint definitions follow the format METHOD /path. Priorities assigned to higher-level requirements are inherited by all sub-requirements unless explicitly stated otherwise.

1.3 Intended Audience and Reading Suggestions

Developers: Focus on Sections 2.5 (Class Diagram), 3.1 (Architectural Design), and 3.3 (Component Design). Database Engineers: Refer primarily to Section 3.2 (Database Design) covering the Data Dictionary and Normalization analysis. Evaluators / Faculty: Read sequentially from Section 1 through Section 3 and the Appendices. Testers: Review Section 2.2 (Use Case Specifications) and Section 3.4 (Interface Design). System Architects: Focus on Section 3.1.1 (System Architecture Diagram) and Section 3.3.3 (Deployment Diagram). It is strongly recommended that all readers follow the document in the order presented for maximum comprehension of the system design.

1.4 References

- [1] React Documentation – <https://react.dev>
- [2] Node.js & Express.js Documentation – <https://expressjs.com>
- [3] MongoDB Documentation – <https://www.mongodb.com/docs>
- [4] FastAPI Documentation – <https://fastapi.tiangolo.com>
- [5] Tailwind CSS Documentation – <https://tailwindcss.com/docs>
- [6] JSON Web Token (JWT) – RFC 7519 – <https://tools.ietf.org/html/rfc7519>
- [7] IEEE Std 1016-2009 – IEEE Standard for Information Technology – Systems Design – Software Design Descriptions.
- [8] Scikit-learn: Machine Learning in Python – <https://scikit-learn.org/stable>

2. Analysis Model

2.1 Methodology Used (Object-Oriented + Agile)

UniCare is developed using an Object-Oriented (OO) design approach combined with the Agile Incremental Software Process Model. The system is structured around clearly defined classes: User, Complaint, Admin, Notification, and AIService. Each class encapsulates its attributes and behaviors, and interactions between components are managed via RESTful APIs exposed by the Node.js + Express backend, with AI classification delegated to a Python FastAPI microservice and all persistent data stored in MongoDB Atlas.

The Object-Oriented approach ensures high cohesion and low coupling across system modules. Inheritance is applied for role-based user entities, while association and aggregation relationships model complaint ownership, admin assignment, and notification delivery. This design promotes modularity, reusability, and maintainability throughout the codebase.

The Software Process Model adopted is the Agile Incremental Model. Development is organized into iterative sprints, each delivering a working software increment: Sprint 1 covers user authentication and role-based access control; Sprint 2 covers complaint submission and the AI classification pipeline; Sprint 3 covers admin resolution workflows and notification dispatch; Sprint 4 covers reporting, analytics dashboard, and UI polish. This model allows continuous feedback from institutional stakeholders and rapid adaptability to evolving requirements.

2.2 Use Case Diagram

The UniCare system supports three primary actors: Student (the primary actor who submits and tracks complaints), Admin (the secondary actor who manages and resolves complaints), and System (the automated actor responsible for AI classification, notification dispatch, and data persistence). Key use cases include: Raise Complaint, Track Complaint Status, View Notifications, Manage Complaint, Assign / Escalate, Generate Reports, and AI Auto-Classification.

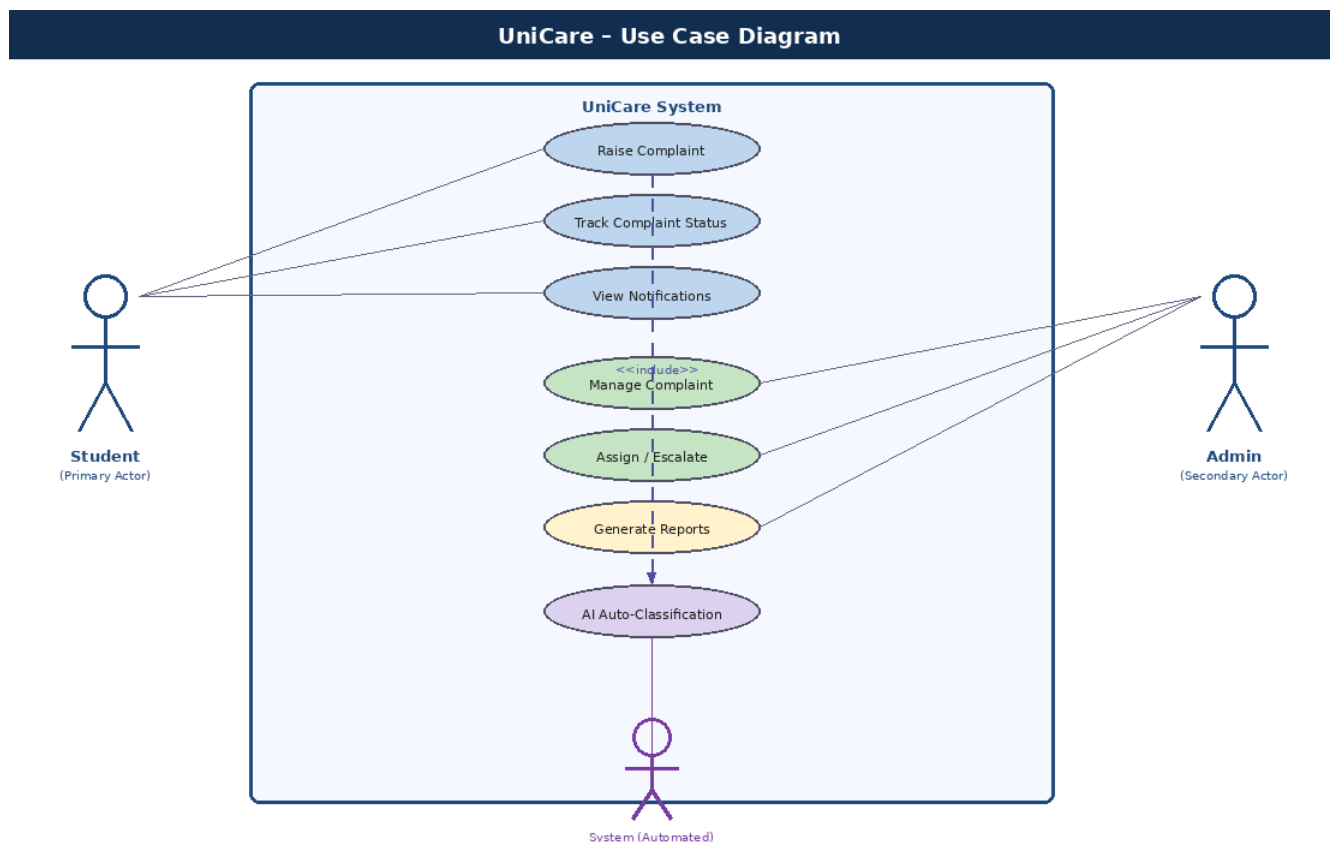


Figure 2.1: UniCare – Use Case Diagram

Use Case Specification 1: Raise Complaint

Name Of Use Case	Raise Complaint
Actor	Student
Pre Condition	Student is authenticated and logged into the UniCare platform. The complaint submission form is accessible from the student dashboard.
Primary Flow of Events	1. Student navigates to the Raise Complaint page from the dashboard. 2. Student fills in the complaint title, description, and optionally selects a category. 3. System validates all input fields for completeness and format. 4. System forwards the complaint text to the FastAPI AI Service for NLP classification. 5. AI Service returns a predicted category and priority level (Low / Medium / High / Critical). 6. System saves the complaint record to MongoDB with all metadata and a status of OPEN. 7. System sends an email and in-app notification to the relevant Admin. 8. System returns a unique Complaint ID and displays a confirmation message to the student.
Alternate Flow Of Events	Alt Flow 1: Validation fails (empty fields or text too short) – System highlights erroneous fields and prompts the student to correct them before resubmission. Alt Flow 2: AI Service is unavailable – System saves the complaint with category UNCLASSIFIED and priority MEDIUM, flagging it for manual Admin review. Alt Flow 3: Network timeout during submission – System displays an error toast and retains form data for re-submission without loss.
Post Condition	A new complaint record is created and stored in MongoDB with status OPEN. The assigned Admin receives a notification alert on the admin dashboard and via email.
Use Case Termination	Success: Student receives a confirmation message with the Complaint ID and estimated response time. Failure: Form errors are displayed inline; no complaint record is created in the database.

Use Case Specification 2: Track Complaint

Name Of Use Case	Track Complaint
Actor	Student
Pre Condition	Student is authenticated. At least one complaint has been previously submitted by the student and a Complaint ID has been issued by the system.
Primary Flow of Events	1. Student navigates to the My Complaints section on the dashboard. 2. System queries MongoDB and displays a paginated list of all complaints submitted by the student. 3. Student selects a specific complaint to view its detail page. 4. System displays full complaint details: title, description, category, priority, current status, assigned admin, and resolution timeline. 5. Student can view all historical status changes and any admin resolution notes logged against the complaint.
Alternate Flow Of Events	Alt Flow 1: Student enters an invalid or non-existent Complaint ID – System displays a not found error message. Alt Flow 2: Student has no complaints on record – System displays an empty state UI with a prompt to raise a new complaint.
Post Condition	No data is modified. Student has a read-only view of the complaint details and its current resolution status.
Use Case Termination	Success: Student views the real-time status and timeline of the selected complaint. Failure: Complaint ID does not exist; an error message is displayed and no data is returned.

Use Case Specification 3: Manage Complaint

Name Of Use Case	Manage Complaint
Actor	Admin
Pre Condition	Admin is authenticated with administrative privileges. One or more complaints exist in the system with status OPEN or IN PROGRESS.
Primary Flow of Events	1. Admin logs into the UniCare Admin Panel and views the complaint queue sorted by priority. 2. System retrieves all open and in-progress complaints from MongoDB. 3. Admin selects a complaint to review its full details including AI-assigned category and priority. 4. Admin takes one of the following actions: Resolve (marks complaint as RESOLVED and adds resolution notes), Reassign (forwards to another admin or department), or Escalate (raises priority to CRITICAL and flags for urgent attention). 5. System updates the complaint record in MongoDB with the new status and timestamps. 6. System sends a notification to the Student informing them of the status change and any resolution notes.
Alternate Flow Of Events	Alt Flow 1: Admin disputes the AI-assigned category – Admin manually overrides the category before resolving. Alt Flow 2: Admin escalates a complaint – System changes priority to CRITICAL and triggers an additional notification to a senior administrator. Alt Flow 3: Complaint has already been resolved by another admin – System displays a conflict warning and refreshes the complaint view.
Post Condition	Complaint status is updated in MongoDB. The student is notified of the resolution or status change. The complaint is removed from the active Admin queue.
Use Case Termination	Success: Complaint is marked resolved; all parties receive confirmation notifications. Failure: Unauthorized access attempt is blocked; a 403 Forbidden response is returned and the event is logged.

2.3 Activity Diagram

The Activity Diagram below illustrates the end-to-end complaint lifecycle in UniCare: the Student logs in and navigates to the submission form; the system validates input (looping back on error); the validated text is forwarded to the FastAPI AI classification service; category and priority are auto-assigned; the complaint is stored in MongoDB; the Admin is notified and takes a management action; and finally the Student receives a resolution notification and the complaint is marked resolved.

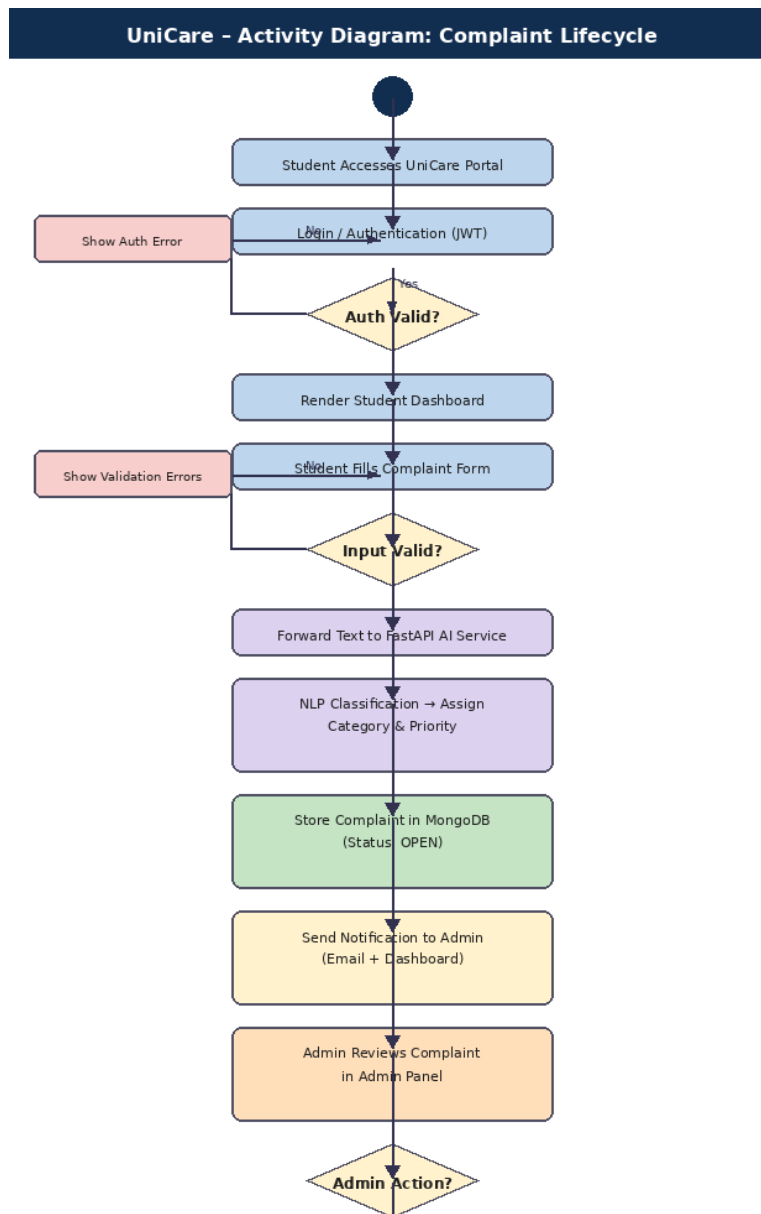


Figure 2.2: UniCare – Activity Diagram: Complaint Lifecycle

2.4 Sequence Diagram

The Sequence Diagram below illustrates the Complaint Submission and Resolution flow across six objects: User (Browser), React Frontend, Node.js Backend, FastAPI AI Service, MongoDB Database, and Admin Panel. The student submits a complaint form; the frontend dispatches a POST request to the backend; the backend forwards the text to the AI service for classification; the classified result is stored in MongoDB; the admin is notified; the admin updates the complaint status; and the student is notified of the resolution outcome.

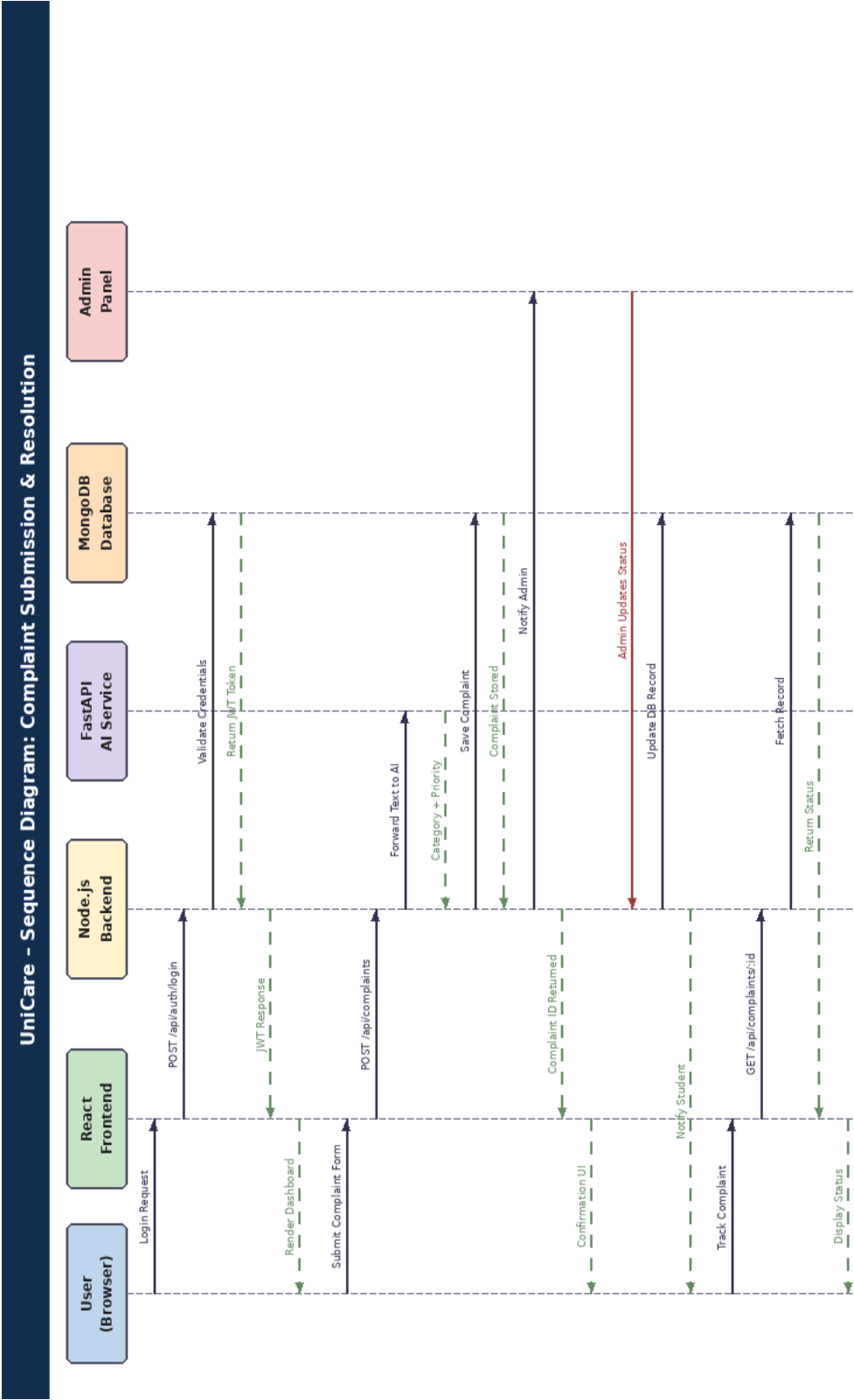


Figure 2.3: UniCare – Sequence Diagram: Complaint Submission & Resolution

2.5 Class Diagram

The Class Diagram below shows five core classes – User, Complaint, Admin, Notification, and AIService – with their attributes, methods, and relationships. A User (1) submits many Complaints (1..*); each Complaint is managed by one Admin; a User receives many Notifications; and the AIService is associated with Complaints through the automated classification process.

UniCare – Class Diagram

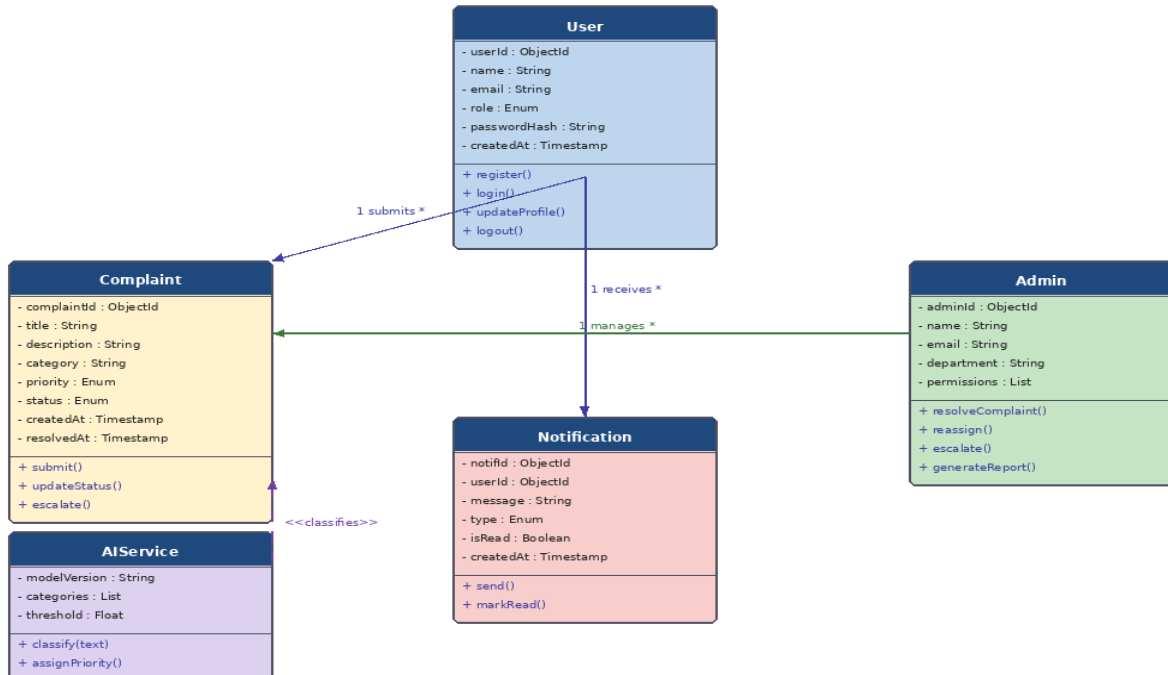


Figure 2.4: UniCare – Class Diagram

3. Design Model

3.1 Architectural Design

3.1.1 System Architecture Diagram

UniCare employs a four-tier layered architecture: Presentation Tier (React.js SPA with Tailwind CSS), Application Tier (Node.js + Express REST API with JWT authentication), Intelligence Tier (Python FastAPI AI Microservice for NLP-based complaint classification), and Data Tier (MongoDB Atlas as the primary NoSQL database). All inter-tier communication occurs over HTTPS using RESTful APIs, ensuring a loosely coupled, independently scalable system design.

UniCare – System Architecture Diagram

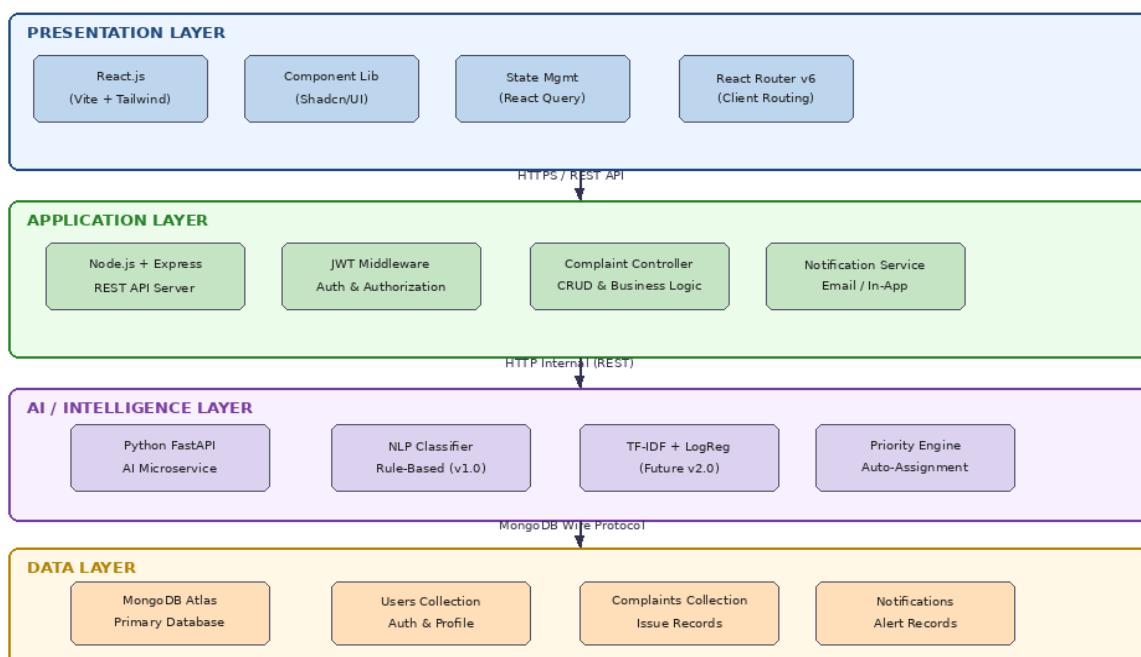


Figure 3.1: UniCare – System Architecture Diagram

3.1.2 Description of Architectural Design

Client Layer: The presentation tier is a React.js single-page application (SPA) built with Vite and styled using Tailwind CSS. The client communicates with the backend exclusively through RESTful HTTPS API calls. React Router v6 manages client-side navigation, and React Query handles data fetching, caching, and synchronization. Role-based UI rendering ensures that Students and Admins see contextually appropriate, distinct interface views. No business logic is executed on the client side.

Application Layer: The application tier is a Node.js + Express.js REST API server responsible for user authentication (JWT-based stateless tokens), complaint CRUD operations, business logic enforcement, and notification dispatch. JWT middleware validates all protected routes and extracts the user role for authorization decisions. The Complaint Controller orchestrates the workflow between the client request, AI service invocation, and database persistence. A dedicated Notification Service dispatches email alerts and in-app notifications upon complaint creation, status change, and resolution.

AI / Intelligence Layer: A Python FastAPI microservice handles all AI inference tasks as an independently deployable service. The Version 1.0 implementation uses rule-based NLP with keyword matching and regular expressions to classify complaints into predefined categories (Academic, Infrastructure, Administration, Hostel, Other) and assign a priority level. Future Version 2.0 will replace this with a trained TF-IDF vectorizer combined with a multi-class Logistic Regression classifier, improving classification accuracy on real institutional complaint datasets.

Data Layer: MongoDB Atlas serves as the primary data store, with separate collections for Users, Complaints, and Notifications. MongoDB's flexible document model accommodates the variable structure of complaint data and supports efficient querying by status, category, priority, and user ID using compound indexes. Mongoose ODM is used by the Node.js application layer for schema definition, data validation, and query building.

3.2 Database Design

3.2.1 Data Dictionary

The following table defines the primary data elements used across the UniCare system collections in MongoDB Atlas:

Name	Alias	Where Used	Content Description	Type / Notes
user_id	uid	All Modules	MongoDB ObjectId primary key for each registered user	ObjectId, NOT NULL, PK
name	full_name	User, Auth	Full name of the registered student or administrator	String, NOT NULL
email	user_email	Auth, Notification	Unique institutional email address used for login and notifications	String, UNIQUE, NOT NULL
role	user_type	Auth, Complaint	Enumerated role of the user: Student or Admin	Enum {Student, Admin}
password_hash	pwd	Auth Module	Bcrypt-hashed password string; plaintext is never stored	String, NOT NULL
complaint_id	cid	Complaint, Notification	MongoDB ObjectId primary key for each submitted complaint	ObjectId, NOT NULL, PK
title	comp_title	Complaint	Short descriptive title of the complaint submitted by the student	String, NOT NULL, max:200

Name	Alias	Where Used	Content Description	Type / Notes
description	comp_desc	Complaint, AI Service	Full text description of the complaint forwarded to the AI classifier	String, NOT NULL, max:2000
category	comp_cat	Complaint, AI Service	AI-assigned category: Academic, Infrastructure, Hostel, Administration, Other	String, DEFAULT: Unclassified
priority	comp_pri	Complaint, Admin Panel	AI-assigned priority level used for Admin queue ordering	Enum {Low, Medium, High, Critical}
status	comp_status	Complaint, Admin Panel	Current resolution status of the complaint record	Enum {Open, In Progress, Resolved, Closed}
created_at	created_ts	Complaint, Notification	ISO 8601 timestamp recorded at the moment of complaint submission	Timestamp, DEFAULT: NOW()
resolved_at	resolved_ts	Complaint	Timestamp recorded when Admin marks the complaint as Resolved	Timestamp, NULLABLE
admin_id	assigned_to	Complaint, Admin Panel	ObjectId of the Admin assigned to handle this complaint	ObjectId, FK → Users
notif_id	nid	Notification	MongoDB ObjectId primary key for each notification record	ObjectId, NOT NULL, PK
message	notif_msg	Notification	Human-readable notification message delivered to the user	String, NOT NULL
is_read	read_flag	Notification	Boolean flag indicating whether the user has viewed the notification	Boolean, DEFAULT: false

Table 3.1: UniCare Data Dictionary

3.2.2 Normalization

The UniCare MongoDB schema design adheres to the principles of relational normalization up to Third Normal Form (3NF), applied within the context of a document-oriented database:

1NF (First Normal Form): All documents within each collection contain atomic values with no repeating groups or nested arrays that duplicate entity data. Each complaint document holds a single, self-contained issue record with a unique ObjectId primary key (complaint_id). The Users, Complaints, and Notifications collections each enforce entity integrity through unique primary keys.

2NF (Second Normal Form): All non-key attributes in each collection are fully functionally dependent on the collection's primary key. Notification-specific data (message, is_read, created_at, type) is separated into a dedicated Notifications collection rather than embedded within Complaint documents, thereby eliminating partial dependency on the user_id reference key.

3NF (Third Normal Form): No transitive dependencies exist across the schema. Admin details (name, email, department) are not stored within Complaint documents; instead, only the admin_id reference is maintained as a foreign-key pointer to the Users collection. This eliminates the transitive dependency admin_name → admin_id → complaint_id and ensures that administrator information is updated in a single location without cascading changes across complaint records.

3.3 Component Design

3.3.1 Package Diagram

The package diagram below illustrates the six top-level source packages of the UniCare codebase and their dependency relationships: /client (React pages and reusable UI components), /api (Express route handlers and controller definitions), /services (business logic layer decoupled from route handlers), /models (Mongoose schema and model definitions), /ai (the Python FastAPI AI microservice codebase), and /config (environment configuration, middleware registrations, and utility helpers).

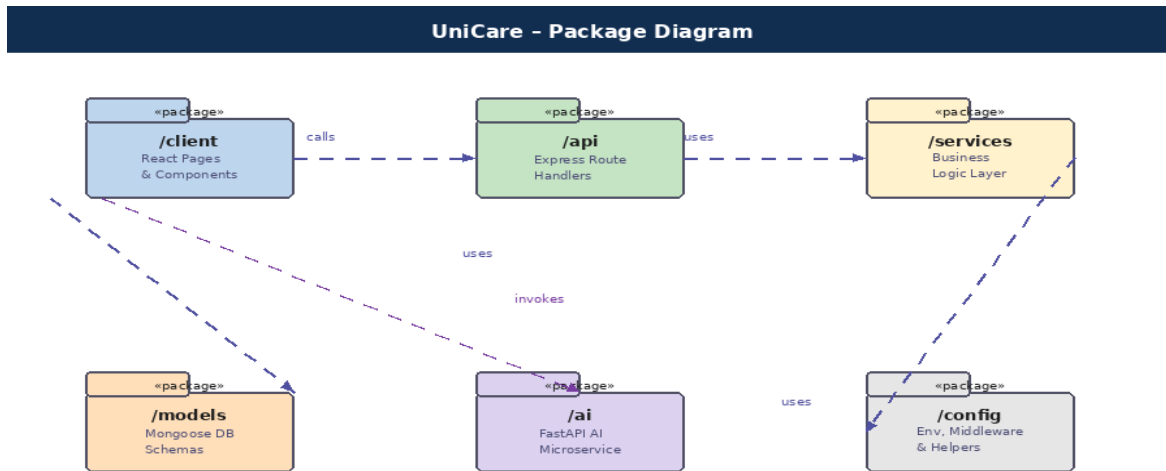


Figure 3.2: UniCare – Package Diagram

3.3.2 Component Diagram

The component diagram below shows six runtime components of the UniCare system: ReactApp, ExpressAPI, MongooseODM, FastAPIService, JWTAuthMiddleware, and NotificationService. The ReactApp communicates with ExpressAPI over HTTP; ExpressAPI delegates authentication verification to JWTAuthMiddleware, data persistence to MongooseODM (connecting to MongoDB Atlas), AI classification to FastAPIService (over HTTP), and notification dispatch to NotificationService. This component separation enforces the single responsibility principle across runtime boundaries and enables independent scaling of the AI tier.

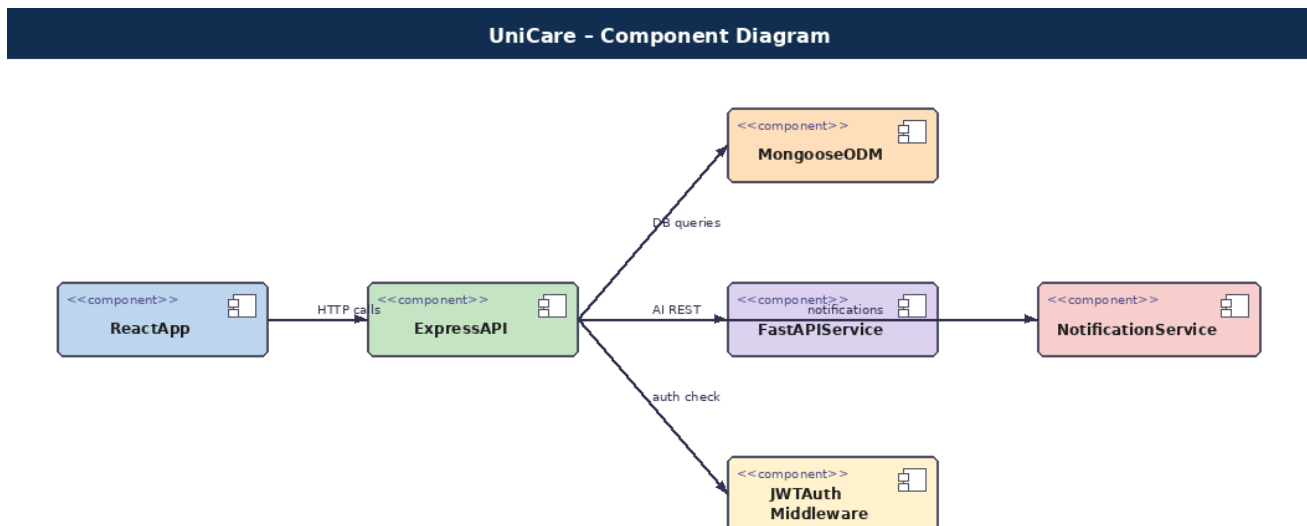


Figure 3.3: UniCare – Component Diagram

3.3.3 Deployment Diagram

UniCare is deployed across four node types: Client Browser (React SPA served via static CDN hosting or Netlify), Application Server (Node.js + Express REST API deployed on Render or Railway Cloud with JWT middleware), AI Microservice Server (Python FastAPI running on Uvicorn, deployed as a separate cloud service on Railway or Fly.io), and MongoDB Atlas (fully managed cloud database cluster). All client-to-server

communication is over HTTPS. Internal service-to-service communication between the Node.js API and the FastAPI microservice occurs over HTTP on private virtual network channels within the cloud provider.

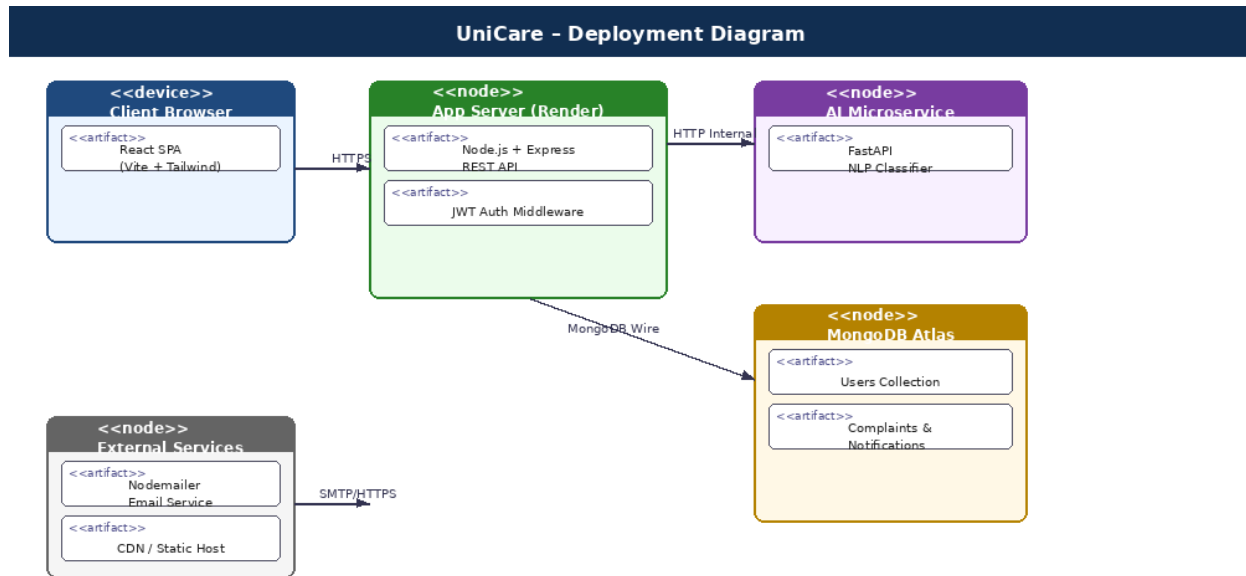


Figure 3.4: UniCare – Deployment Diagram

3.4 Interface Design

3.4.1 Screen Descriptions

The following screens constitute the primary user interface of UniCare. All screens are built using React.js with Tailwind CSS and follow a responsive design pattern compatible with desktop and tablet viewports. Role-based rendering ensures that Students and Admins receive contextually appropriate interface experiences upon authentication.

Screen Name	Description & UI Elements
Login Page	The entry point of the UniCare application. Displays the UniCare logo and platform tagline. Contains an email input field, a password field with show/hide toggle, and a Login button. A JWT token is issued and stored securely upon successful authentication. Includes inline error messaging for invalid credentials and a Forgot Password link. Role-based redirection routes Students to the Student Dashboard and Admins to the Admin Panel.
Student Dashboard	Displays a personalized welcome header with the student's name and enrollment details. A summary metrics bar shows: Total Complaints Raised, Open Complaints, and Resolved Complaints (colour-coded). The main content area renders a paginated table of the student's complaints with columns for Complaint ID, Title, Category, Priority (colour-coded badge: red for Critical, orange for High, yellow for Medium, green for Low), Status chip, and Submission Date. A prominent Raise New Complaint button is positioned in the top-right corner.
Raise Complaint Page	A single-page form accessible from the Student Dashboard. Contains a Title input field (max 200 characters with live character counter), a Description textarea (min 50 / max 2000 characters), and an optional Category dropdown pre-populated with standard options. On submission, an AI Processing spinner displays while the FastAPI service classifies the complaint text. After classification, an inline confirmation panel displays the auto-detected Category and Priority before the student confirms final submission.
Complaint Detail Page	Displays the full details of a selected complaint in read-only mode for students. Structured into sections: Complaint Metadata (ID, Submission Date, Category, Priority Badge), Description Panel (full text), Status Timeline (visual stepper: Open → In Progress → Resolved), Assigned Support Team label (admin identity is anonymized for privacy), and Resolution Notes panel (visible exclusively once the complaint status is Resolved).

Screen Name	Description & UI Elements
Admin Panel	The administrative management interface. A collapsible left sidebar provides navigation: Complaint Queue (default view), All Complaints, Analytics & Reports, and Settings. The main area renders the Complaint Queue: a sortable, filterable data table with columns for Complaint ID, Student Name, Category, AI Priority, Status, and Date Received. Filters for Status, Category, and Priority are available. Three primary action buttons are available per complaint: Resolve, Reassign, and Escalate. All actions update the MongoDB record in real time and dispatch student notifications.

Table 3.2: UniCare Interface Screen Descriptions

Appendix C1: Glossary

Term	Definition
AI (Artificial Intelligence)	The simulation of human intelligence processes by computer systems. In UniCare, AI refers to the automated complaint classification and priority assignment capability delivered by the Python FastAPI microservice.
NLP (Natural Language Processing)	A branch of AI concerned with the interaction between computers and human language. UniCare uses NLP techniques – rule-based in v1.0 and ML-based in future versions – to analyse complaint text and extract category and priority information.
FastAPI	A modern, high-performance Python web framework for building APIs, used in UniCare as the AI microservice host. FastAPI leverages Python type hints for automatic request validation and is powered by Uvicorn (ASGI server).
JWT (JSON Web Token)	An open standard (RFC 7519) for securely transmitting information between parties as a JSON object. UniCare uses JWTs for stateless user authentication; tokens are signed with a server-side secret key and validated on every protected API request by the auth middleware.
MongoDB	A document-oriented NoSQL database that stores data in flexible, JSON-like BSON documents. UniCare uses MongoDB Atlas (cloud-hosted managed cluster) as its primary data store for Users, Complaints, and Notifications collections.
Mongoose ODM	An Object-Document Mapping (ODM) library for Node.js that provides schema-based data modelling for MongoDB. UniCare uses Mongoose for schema definition, field-level validation, and query building within the application layer.
REST API	Representational State Transfer Application Programming Interface. A set of architectural constraints for building stateless, client-server, cacheable web services using standard HTTP methods (GET, POST, PUT, PATCH, DELETE).
TF-IDF	Term Frequency-Inverse Document Frequency. A numerical statistic used in NLP to reflect how important a word is to a document relative to a corpus. Planned for use as the primary feature extraction technique in UniCare's future ML-based complaint classifier.
Logistic Regression	A supervised machine learning algorithm used for binary or multi-class classification. UniCare's future Version 2.0 ML model will use Logistic Regression trained on historical institutional complaint data to predict complaint category with higher accuracy.
React.js	An open-source JavaScript library for building user interfaces, maintained by Meta. UniCare's frontend is implemented as a React single-page application (SPA) using Vite as the build tool and Tailwind CSS for styling.
Tailwind CSS	A utility-first CSS framework that provides low-level styling classes for building custom interfaces directly in HTML/JSEX markup. Used throughout the UniCare frontend to implement a consistent, responsive design system.
Node.js + Express	Node.js is a server-side JavaScript runtime environment; Express.js is a minimal, unopinionated web application framework for Node. Together they form the UniCare application-tier REST API server.

Term	Definition
ObjectId	A MongoDB-specific 12-byte identifier (represented as a 24-character hexadecimal string) automatically generated by MongoDB for each document, serving as the primary key. Used as complaint_id, user_id, and notif_id throughout UniCare collections.
3NF (Third Normal Form)	The level of relational database normalization applied to UniCare's schema design, ensuring no transitive dependencies exist among non-key attributes within any collection.
CORS (Cross-Origin Resource Sharing)	An HTTP-header-based security mechanism that allows a server to specify which origins a browser is permitted to load resources from. Configured in UniCare's Express server to allow requests exclusively from the authorized React frontend origin.

Table C1: UniCare Glossary of Technical Terms

Appendix C2: To Be Determined List

TBD ID	Area	Description
TBD-01	AI / ML Model Upgrade	The current rule-based NLP classifier must be replaced with a trained machine learning model using TF-IDF vectorization and multi-class Logistic Regression. The training dataset curation strategy, minimum corpus size for acceptable accuracy, model retraining schedule, and versioning approach are yet to be finalized. Minimum acceptable F1 score thresholds per category must be defined before the ML model is deployed to production.
TBD-02	JWT Secret Management & Rotation	The mechanism for secure storage and automatic rotation of the JWT signing secret has not been finalized. Options under evaluation include environment variable injection (current approach), integration with a dedicated secrets manager (HashiCorp Vault or AWS Secrets Manager), and an automatic key rotation policy with graceful token invalidation and re-issuance strategy.
TBD-03	Email Notification Service Provider	The production email service provider for complaint submission confirmation, status update, and resolution notifications has not been selected. Candidates include SendGrid, Mailgun, and Amazon SES. The selection will be based on institutional email domain configuration requirements, monthly volume quotas, deliverability guarantees, and cost.
TBD-04	Admin Role Hierarchy & Escalation Routing	The organizational hierarchy for Admin roles within UniCare (e.g., Department-Level Admin, Dean-Level Admin, Registrar) and the automated escalation routing logic between department-level and institution-level administrators requires formal definition in consultation with institutional management and stakeholders before implementation.
TBD-05	Complaint SLA & Auto-Escalation Triggers	Service Level Agreement (SLA) thresholds for complaint acknowledgement and resolution times (e.g., High priority complaints must be acknowledged within 24 hours and resolved within 72 hours) and the automated escalation triggers are yet to be formally agreed upon with institutional administration.
TBD-06	Analytics Dashboard Scope & Export Format	The full scope, metrics, chart types, and data visualizations required for the Admin Analytics and Reporting module have not been specified. Pending decisions include the granularity of time-period filters for trend analysis, whether complaint data exports will be provided in CSV or PDF format, the specific KPIs to be tracked, and whether real-time streaming analytics via MongoDB Change Streams will be required.

Table C2: UniCare To Be Determined List